

Word Count in Python

Mahmoud Parsian

Ph.D. in Computer Science

What is the Word Count Problem?

- **Input:** a set of text documents
- **Program:** counts the number of occurrences of each unique word
- **Output:** (word, frequency) pairs

Example

Input:

```
"fox jumped over red fox over fox jumped"
```

Output:

fox	3
jumped	2
over	2
red	1

A First Solution: In-Memory String

```
def word_count(str):  
    counts = dict()  
    words = str.split()  
  
    for word in words:  
        if word in counts:  
            counts[word] += 1  
        else:  
            counts[word] = 1  
        #end-if  
    #end-for  
  
    return counts  
#end-def
```

Sample Run

```
>>> print(word_count(  
...     'fox jumped and jumped a gray fox jumped and jumped'))  
{'and': 2, 'jumped': 4, 'fox': 2, 'a': 1, 'gray': 1}
```

`split()` + bump a counter in a `dict` + return it — that's the whole algorithm. Everything else in this deck (and the MapReduce version) is this same idea, adapted to a different input source or execution model.

From a String to Real Files

Realistically the input is a file (or a directory of files), not a string literal. The up-to-date, runnable versions of that step already live in this repo:

[word_count_in_python/](#)

Script	What it does
<code>word_count_single_file.py</code>	One text file → counts → stdout
<code>word_count_single_file_v2.py</code>	Improved version of the above
<code>word_count_dir_to_tsv.py</code>	A directory of <code>.txt</code> files → one TSV
<code>word_count_dir_to_tsv_with_filter.py</code>	Same, + min length / min frequency filters

A Real Run, Not a Hypothetical One

```
% python3 word_count_single_file_v2.py test_file.txt
input_file= test_file.txt
and: 3
cute: 1
fox: 8
gray: 2
is: 1
jumped: 7
over: 3
red: 3
```

Same `split()` + count-in-a- `dict()` idea as `word_count()` above — this is that idea, actually run against a real file. Full output (including the directory/TSV and filtered variants) in [word_count_in_python/README.md](https://github.com/mahmoudp/word-count-in-python/blob/master/README.md).

Limitations of Word Count in Plain Python

1. All algorithmic steps are sequential
2. The program can only run on a single computer
3. Data size is limited to that one computer's disk space
4. Cannot handle big data (billions of records)
5. The solution does not scale out

Overcoming These Limitations

MapReduce/Hadoop (slow)

- Uses disk I/O between stages
- Comparatively hard to write jobs

Spark/PySpark (fast)

- Uses memory/RAM as much as possible, disk I/O as a fallback
- A superset of MapReduce
- Comparatively easy to write jobs

Next

The same `split()` + count-in-a-dictionary idea from this deck, worked out as a MapReduce job — partitioned across mappers and reducers instead of one sequential loop:

[04_word_count_in_mapreduce.md](#)